Computing Science Background Package

TeamUp Science ISC 2026

```
main.py                                                    UTF-8

Hello there!!
Welcome to ISC 2026 ^◡^
```

Table of Contents

1. Learning Objectives & Tips

By participating in the ISC 2024 Computer Science Lab, students will practice writing Python code to create a practical, functional program and learn the following concepts:

- Understand basic data types and when to use them
- Assign variables
- Use I/O methods to interact with the console
- Use conditionals and loops
- Understand basic data structures (especially arrays and dictionaries) and their practical applications
- Use loops to traverse an array or dictionary
- Create functions
- Comment out their code and use pseudocode

Some extra things they can learn are:

- Scope
- Classes

Tips!

⇒ In programming, it's important to be able to use your resources well – an important resource all programmers use is the internet! If you do not understand or want to know more about a concept, make sure to search it up. Be specific and know what you're looking for!
Some helpful websites include:

- Python Website
- GeeksforGeeks
- RealPython
- W3Schools
- Stack Overflow

⇒ Write it out: before tackling a problem, it's a good idea to map out your approach to not only organize your thoughts, but gain a better understanding of what you need to do.

- Also very helpful to write pseudocode (we cover *pseudocode* in Section 10)

⇒ Test as you go: when building programs, it's helpful to break it down into smaller sections and test that your code works as you go along rather than testing once you've built everything. This way it's easier to fix your mistakes and also leads you to arriving at the correct solution.

Beginner Package

2. Variables

Variables are like labeled boxes: they store data and give it a name so you can easily access it later. Essentially, a variable is a place in computer memory with an assigned name and value.

- We need variables to store and retrieve data of different types (text, numbers, etc.).

Creating a variable is called **'declaring the variable'**. You name your variable and assign a value to it in the process of assignment.

```
# Example of variables
age = 16  # 'age' is a variable storing the value 16
name = "Alice"  # 'name' is a variable storing the string "Alice"
```

You can also change the content of a variable as needed during your program.

```
x = 4       # x has the value 4
print(x)
x = "Sally" # x now has the value “Sally”
print(x)

// Output
>> 4
>> Sally
```

Now you may be wondering if there are limitations to how we can name these variables.

Here are some rules for naming variables:

1) You can use **letters, digits, and underscores (_)** in a variable name.
2) Variable names are **case sensitive**.
    - *example, Example, and eXampLe* are all different variables as far as Python is concerned.
3) You can't start a variable name with a **digit**.
    - Ex: 1team = “Red” – This is illegal!
4) You can't use a **reserved word** (i.e. def, if, else, False, True, None...) as a variable name.
    - If you're not sure what words are reserved (i.e. Python uses them for certain functions), type help('keywords') into your shell.
5) You assign a value to a variable **using the equals sign** (=). First comes the variable name, then the equals sign, then the value:
    - favourite_food = 'ice cream!'

3. Data Types

Variables can store **data of different types**, and different types can do different things.
In Python, there are several data types which are listed below. Each type serves a **unique** purpose, allowing your program to handle various kinds of information.

Data Type	***Definition***
int (Integer)	Whole numbers which can be negative or positive. - Ex: -2, 0, 42.
str (String)	Strings are sequences of characters enclosed in quotes (either " or '). - Ex: "My name is Ye!", "apple bottom jeans"
bool (Boolean)	Booleans can be only one of two values: True or False. - Ex: is_sunny = True, is_rainy = False
float (Floating-point)	Floating-point numbers include decimals (like fractions). - Ex: 80.08, 3.14159

```
# Examples of data types
age = 16  # int
name = "Alice"  # str
is_student = True  # bool
height = 5.6  # float
```

You can get the data type of any object by using the type() method (a built-in Python function):

```
# Examples of type from above
age_type = type(age)
name_type = type(name)

print(age_type)
print(name_type)

// Output
>> int
>> str
```

4. I/O Methods

Let's look at how your program works through Input and Output methods.

The print() function is your computer's way of talking to you. It lets you display information on the screen.

```
# Example of print()
name = "Cudi"
print("Hello, " + name + "!")

// Output
>> Hello Cudi!
```

The + sign here is like gluing different pieces of information together before displaying.

Notice how the quotation marks do not get printed. This is because they serve the purpose of telling the computer that you are using a string – it is NOT a symbol to be printed.

- If you want to print quotation marks, you can use the other type of quotation marks to enclose your string.

```
# If we want to print “
name = “Lil Tecca”
print(‘Hello my “name” is’ + name + ‘!’)

# If we want to print ‘
name = “Tyler”
print(“Is that Tyler’s new song?”)

// Output
>> Hello my “name” is Lil Tecca
>> Is that Tyler’s new song?
```

Just make sure that the surrounding/enclosing pair of quotation marks **match**!
So, either use ‘ or “ but don’t mix & match!

The input() function is your way of talking to the computer. It allows you (or anybody using the program) to type information that the program can use.

```
# Example of input()
age = input("How old are you?")          # “How old are you?” will be printed as an input prompt
print("You are " + age + " years old.")

// Output
>> How old are you?
>> 17
>> You are 17 years old.
```

As you can see, input() captures what you type after printing the prompt included inside the input() call and stores the captured value for the program to use later.

5. Conditional Statements

Suppose a program needs to skip over some statements, execute a series of statements repetitively, or choose between alternate sets of statements to execute – in these situations, we use a control structure.

- A control structure directs the order of execution of the statements in a program (referred to as the program's control flow).

We make this control structure using conditional statements which allow your program to make decisions based on certain conditions.

If/Elif/Else Statements

The if statement is the most basic form: it executes a block of code **only if** a specified condition is **true**.

```
# Example of an if statement
age = 18

if age >= 18:
    print("You are eligible to vote.")
```

Here, the code inside the if block will only be executed if the condition age >= 18 is true.

An else statement executes a different block of code when the condition is **false**.

```
# Example of an if-else statement
age = 15

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote yet.")
```

Here, if the condition age >= 18 is false, the code inside the else block will be executed.

If there's multiple conditions to check, you can use elif (short for else if) to add more conditions.

```
# Example of an if-elif-else statement
grade = 85

if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")
elif grade >= 70:
    print("C")
else:
    print("Below C")
```

This construct allows you to check multiple conditions in order. Once a condition is true, the corresponding block of code is executed, and the rest of the conditions are skipped.

While Loop

While loops are used when you want to **repeat a block of code** as long as a condition is true.

- While loops are very helpful when you **don't know how many times** you want to repeat a block of code BUT have a specific condition that, when met, stops the loop.

```
# Example of a while loop with a conditional statement
count = 0

while count < 5:              # This while loop stops when count becomes 5 or greater
    print(count)
    count += 1                        # This statement increments the count for each iteration
```

This loop will print numbers from 0 to 4 as long as the condition count **< 5** is true. The count += 1 statement increments the count in each iteration.

Conditional statements and loops are powerful tools in programming. They allow your program to adapt to different situations and handle various scenarios.

---

6. Lists

A list is like a **dynamic container** that can hold multiple items. Each item is called an element.

- Lists are incredibly versatile and can store **various** data types, meaning you can store a mix of ints, strings, and floats in one list.

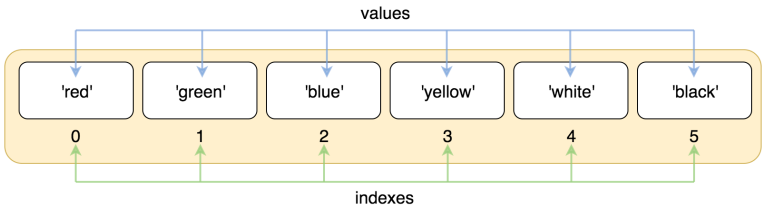
We make a list by enclosing items in square brackets **[ ]**, separated by commas.
Here are a few ways to initialize a list:

```
# Example: Initializing a list
fruits = ['apple', 'banana', 'orange']
numbers = [1, 2, 3, 4, 5]
mixed_types = ['apple', 123, 3.14, True]
empty_list = []
```

In this example,
→ List of Fruits: A list of strings representing fruit names.
→ List of Numbers: A list of integers.
→ Mixed Types List: A list can contain elements of different data types.
→ Empty List: You can initialize an empty list with [] (ie. empty brackets).

Elements in a list are **accessed** using their indices (singular; index), which **starts at 0** (**NOT 1).

- This means the first item has an index of 0, the second item has an index of 1, the third item has an index of 2 and so on...



This process of accessing elements from the list is called indexing, which means we call them using their position on the list.

- Square brackets are used to access elements by their index.
- The syntax for accessing an element is: ***listName[elementIndex]***

There is also **negative indexing** which starts from **-1, -2, -3, and so on** — this allows you to access the list from the other end and can be helpful in getting the last element in a list.

- The last element in a list has an index of -1.

```
# Example: Indexing a list / getting elements from a list
fruits = ['apple', 'banana', 'orange']

first_fruit = fruits[0]  # This will be "apple"
second_fruit = fruits[1]  # This will be "banana"
third_fruit = fruits[2] # This will be “orange”
neg_index_fruit = fruits[-1] # This will be “orange” – notice how fruits[-1] is the same as fruits[2]

print(first_fruit)
print(second_fruit)
print(third_fruit)
print(neg_index_fruit)

// Output
>> apple
>> banana
>> orange
>> orange
```

You can extract **a portion** of a list using slicing.

- The syntax is ***listName[start:stop]***
    - where start is the inclusive index, and stop is the exclusive index

```
# Slicing lists
fruits = ['apple', 'banana', 'orange']
selected_fruits = fruits[1:3]
print(selected_fruits)

// Output
>> [‘banana’, ‘orange’]
```

Lists are **mutable**, meaning you can change their contents after creation.

```
# Modifying a list
fruits = ['apple', 'banana', 'orange']
fruits[2] = "pear"  # Changes "orange" to "pear"

# fruits is now: fruits = [‘apple’, ‘banana’, ‘pear’]
```

7. Dictionaries

A dictionary is like a real-life dictionary with keys and values.
Keys act as the words, and values act as the definitions.

- Each key is **unique**, and it corresponds to a specific value.
- They are very useful when you want to quickly look up information.

Unlike lists, dictionaries consist of key-value pairs, NOT single values.
Key-value pairs look like this: **'Key': 'Value'**
or
'Country': 'Capital city'

To access dictionary values, we use keys rather than indexes. Because of that, dictionary keys are **unique** and **immutable** – they cannot be changed. The values stored in dictionaries can be of any data type, including other dictionaries (ie. a nested dictionary).

```
# Example of a dictionary
capitals = {'Czechia':'Prague', 'Lithuania': 'Vilnius', 'France':'Paris', 'Germany':'Berlin',
'Italy':'Rome', 'Poland':'Warsaw'}
```

To access one of the values, we need to write the chosen key in square brackets:
Take a look at an example of accessing the capital of Italy from our dictionary above:

```
# Accessing a dictionary value
print(capitals['Italy'])

// Output
>> Rome
```

You can also edit dictionaries like adding or removing key-value pairs.

```
capitals['Canada'] = 'Ottawa'       # Adding a new key-value pair to our dictionary of capitals
capitals['France'] = 'Not Paris'    # Changes France's capital to 'Not Paris' instead of 'Paris'
```

Python has some built-in functions, which we call methods, that we can use for dictionaries.
Here we will list some that may be useful to you…
keys() returns a list containing the dictionary's keys – to use: ***dictionaryName.keys()***

```
# Using keys() with a dictionary
all_capitals = capitals.keys()   # returns a list of all country names in our dictionary (ie. the keys)

print(all_capitals)

// Output
>> ['Czechia', 'Lithuania', 'France', 'Germany', 'Italy', 'Poland', 'Canada']
```

values() returns a list containing the dictionary's values – to use: ***dictionaryName.values()***

```
# Using values() with a dictionary
all_capital_countries = capitals.values()  # returns a list of all country's capitals (ie. the values)

print(all_capitals_countries)

// Output
>> ['Prague', 'Vilnius', 'Not Paris, "Berlin', 'Rome', 'Warsaw', 'Ottawa']
```

You can also **check if a key is in the dictionary** by using the keyword *in*

```
# Checking if a key is in the dictionary
is_france_in_capitals = "France" in capitals  # This will be True
is_india_in_capitals = "India" in capitals  # This will be False

print(is_india_in_capitals)

// Output
>> False
```

pop() removes specific key-value pairs in the dictionary – to use: ***dictionaryName.pop(keyName)***

```
# Removing a country from our capitals dictionary
capitals.pop('Lithuania')       # we are removing the key-value pair with Lithuania as its key
print(capitals)

// Output
>> {'Czechia':'Prague', 'France':'Not Paris', 'Germany':'Berlin', 'Italy':'Rome', 'Poland':'Warsaw',
'Canada':'Ottawa'}
```

If we want to save the value that we removed using the key in *pop*, we can save the call to a variable for later use:

```
# Saving the value we removed to a variable
removed_capital = capitals.pop('Lithuania')     # removing the key-value pair with Lithuania as key
print(removed_capital)

// Output
>> 'Vilnius'
```

To clear a dictionary we can use clear() – to use: ***dictionaryName.clear()***

```
# Clears a dictionary
capitals.clear()     # removes all key-value pairs from the dictionary
print(capitals)

// Output
>> {}
```

8. Loops

Loops allow you to **repeat a set of instructions multiple times** without writing the same code over and over again. In Python, two common types of loops are for loops and while loops.

Understanding For Loops

For loops are like having a robot perform a task multiple times. For loops are perfect when you know in advance **how many times** you want to **repeat** a block of code. Let's see how they work:

```
# Example of a for loop
fruits = ["apple", "banana", "orange"]

for fruit in fruits:
   print(fruit)

// Output
>> apple
>> banana
>> orange
```

In this example, the loop iterates through each item in the fruits list, and fruit takes on the value of each item one by one. The print(fruit) statement is then executed for each iteration, printing each fruit on a new line.

Range Function — to use: ***syntax: range(start, stop, step)***

The range() function is often used with for loops to generate a sequence of numbers. It returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

- Remember, Python uses 0-based indexing meaning it counts starting from 0.

```
# Using range() with a for loop
for i in range(5):
   print(i)

// Output
>> 0
>> 1
>> 2
>> 3
>> 4
```

This loop will print numbers from 0 to 4. The range(5) generates a sequence of numbers starting from 0 and ending at 4.

- In range(5), the range of 5 numbers it prints starts at 0 then goes til 4.

Break

Sometimes you want to exit a loop early. We do this using the break statement.

```
# Using break in a loop
fruits = ["apple", "banana", "orange"]

for fruit in fruits:
    if fruit == "banana":
        break
    print(fruit)

// Output
>> apple
```

This loop will print "apple" and then stop when it encounters "banana."

Continue

If you want to skip the rest of the code in the loop for a particular iteration and move to the next one, you can use the continue statement.

```
# Using continue in a loop
fruits = ["apple", "banana", "orange"]

for fruit in fruits:
    if fruit == "banana":
        continue
    print(fruit)

// Output
>> apple
>> orange
```

This loop will print "apple" and "orange" but skips the iteration for "banana."

While Loop

We covered while loops in 5: Conditional Statements

Though, if you still need some more clarification on how while loops work, here is a more in-depth explanation:

A while loop is a control flow statement that allows you to repeatedly execute a block of code as long as a certain condition is true. The basic syntax of a while loop looks like this:

```
while condition:
    # code to be executed while the condition is true
    # this code will repeat as long as the condition remains true
```

Here's a breakdown of how a while loop works:

1) **Initialization**: You start by initializing a variable or using an existing variable that represents the condition.
2) **Condition**: The loop will continue to execute as long as the specified condition is true. If the condition becomes false, the loop will exit, and the program will move on to the next section of code after the loop.
3) **Code Block**: The indented block of code below the while statement is the body of the loop. This code will be executed repeatedly as long as the condition is true.
4) **Update the Condition**: Inside the loop, you typically have some code that updates the variable used in the condition.
    - If you forget to update the condition and it remains true, the loop will run indefinitely, leading to what is called an "infinite loop."

```
# Initialize a variable
count = 1

# Set the condition for the while loop
while count <= 5:
    # Code to be executed while the condition is true
    print(count)

    # Update the condition by incrementing the variable
    count += 1

# The loop will exit when count becomes greater than 5

// Output
>> 1
>> 2
>> 3
>> 4
>> 5
```

In this example, the loop will run as long as the count variable is less than or equal to 5. The loop prints the current value of count and increments it by 1 in each iteration. Once count becomes 6, the condition becomes false, and the loop exits.

9. Functions

A function is a reusable block of code that performs a certain action (or actions). They are used to **accomplish the same tasks repeatedly**.

- Instead of copy-pasting the same lines multiple places in your code, you should use a function.
- Functions also divide complex code into smaller parts, which make it far more readable and maintainable.

Python has several useful built-in functions. To use a built-in function, you don't need to write all the code yourself; just call the function.

If we listed all of the built-in Python functions here it would be way too long so here is a helpful link that contains a run-down of most (if not all) of the built-in Python functions. Remember, the internet is your best friend in the computing science world so if you're curious or want to know more about anything, just look it up!

Link to Python Built-In Functions

Writing Functions

Of course, you don't have to limit yourself to built-in functions. We can create our own.

1. First, we need the *def* keyword.
    - This stands for "**define**" and lets Python know this is going to be a new function.
    - To define a function in Python, you use the def keyword, followed by the function name and a pair of parentheses.
2. We can also pass information to our function for us to use within the function. Information can be passed into functions as arguments.
    - Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.
    - The terms parameter and argument can be used for the same thing: information that is passed into a function.
        - From a function's perspective:
          A parameter is the variable listed inside the parentheses in the function definition.
          An argument is the value that is sent to the function when it is called.
3. Then we need a meaningful name, followed by some code (which is called the 'body' of the function).
    - The function body is **indented** beneath the def statement.
    - Indentation tells the interpreter that this code belongs to this function. Also, note that after the end of the function body, we need to leave an empty line.

```
# A general structure for writing your own function
def function_name(parameter):
    your code
```

Let's try writing our own function! In this example, greet is the function name, and it takes one parameter (name). The function body prints a greeting message.

```
# Here is an example of our own greeting function
def greet(name):
    print("Hello, " + name + "!")        # notice the indentation in our function body
```

To use (call) a function, you simply write its name followed by parentheses. If the function takes parameters, you provide the values inside the parentheses.

```
# Calling our greet function
greet("Alice")

// Output
>> Hello, Alice!
```

Functions can also return values using the return statement. The value can then be assigned to a variable or used in other parts of your code.

```
# Adding two numbers together
def add_numbers(a, b):
    return a + b

result = add_numbers(3, 5)
print(result)

// Output
>> 8
```

In this example, the add_numbers function takes two parameters (a and b) and returns their sum.

You can also provide **default values** for function parameters. Say your function requires a parameter when it's called. If the parameter value is **not provided** when calling the function, the default value is used.

```
def greet(name="Guest"):
    print("Hello, " + name + "!")

greet()
greet("Bob")

// Output
>> Hello, Guest!
>> Hello, Bob!
```

Notice how in our example above, the first call to our greet function used the default value of "Guest" when we did not provide a name.

10. Comments and Pseudocode

Comments

Comments in Python are text annotations within the code that are ignored by the Python interpreter.

- Comments are preceded by the # symbol.
- Comments are like notes in your textbook and are used to add explanations, notes or reminders about your code. They help you and others understand your code better when you review them for later.

```
# This is a single-line comment
print("Hello, World!")  # This is also a comment at the end of a line
```

While Python doesn't have a specific syntax for multiline comments, you can use (single or double) triple quotes (''' or """) to create multiline strings, effectively achieving the same result.

```
'''
This is a multiline comment
It spans multiple lines
'''
print("Hello, World!")
```

Why are comments useful?

1) **Documentation**: Comments serve as a form of documentation, helping you and others understand the purpose and functionality of different parts of the code.
2) **Debugging**: Comments can be used to temporarily disable or "comment out" certain lines of code for debugging purposes.
3) **Communication**: When working in a team, comments help communicate ideas, decisions, or warnings to other team members.

Pseudocode

Pseudocode is a high-level description of an algorithm or a program written in plain language, ***without*** the **specific syntax** of a programming language.

→ It's used to outline the logic and steps of a solution before actually implementing it in code.

Ex: Pseudocode for calculating the average of three numbers

1. Get the first number from the user
2. Get the second number from the user
3. Get the third number from the user
4. Calculate the sum of the three numbers
5. Divide the sum by 3 to get the average
6. Display the average

Why is pseudocode useful?

1) **Clarity**: Pseudocode allows you to focus on the algorithm's logic without getting bogged down by the details of a specific programming language.
2) **Planning**: It helps in planning and organizing your thoughts before diving into actual coding.
3) **Collaboration**: Pseudocode can be easily shared and understood by team members, making it a helpful tool for collaborative development.
4) **Problem Solving**: Pseudocode is particularly useful for solving complex problems by breaking them down into smaller, manageable steps.

Intermediate Package

11. Scope

Understanding where you can use your variables is crucial. Let's talk about scope.

In Python, variables can have different scopes.

- Global scope means the variable can be accessed anywhere in your program, while local scope means the variable is only accessible in a specific part of your code.

```
# Global and local scope
x = 10  # This is a global variable

def my_function():
    y = 5  # This is a local variable
    print(x + y)

# Calling the function
my_function()

# Accessing global variable outside the function
print(x)  # This works

# Trying to access local variable outside the function - will result in an error
# print(y)  # Uncommenting this line will give an error
```

In this example, x is a global variable accessible throughout the program. y, defined inside the function my_function, is local to that function. Understanding scope is crucial for organizing your code effectively. Variables defined inside a function are only known within that function.

---

12. F-Strings

F-strings, introduced in Python 3.6, provide a concise and readable way to embed expressions inside string literals. The "f" stands for "formatted," and f-strings offer a more efficient and convenient way to format strings compared to older methods like concatenation or the format() method.

To create an f-string, prefix the string literal with the letter "f" or "F" and place expressions inside curly braces {} within the string.

```
name = "Rocky"
age = 9000
print(f"Hello, my name is {name} and I am {age + 10} years old.")

// Output
>> Hello, my name is Rocky and I am 9000 years old.
```

Here are some examples to show how helpful f-strings can be!

```
# Basic Usage
name = "Milly"
greeting = f"Hello, {name}!"
print(greeting)

// Output
>> Hello, Milly!
```

```
# Expressions
num1 = 10
num2 = 5
result = f"The sum of {num1} and {num2} is {num1 + num2}."
print(result)

// Output
>> The sum of 5 and 10 is 15.
```

```
# Formatting numbers
pi = 3.141592653589793
formatted_pi = f"Value of pi: {pi:.2f}"
print(formatted_pi)

// Output
>> Value of pi: 3.14
```

```
# Using Variables and Expressions
quantity = 3
price_per_unit = 25.50
total_cost = f"Total cost: ${quantity * price_per_unit:.2f}"
print(total_cost)

// Output
>> Total cost: $76.50
```

---

13. Classes

Classes are like blueprints for creating objects, bringing a new level of organization and structure to your code.

An object is **an instance of a class**, and each object created from a class can have its own unique attributes (characteristics) and methods (functions). Classes provide a way to model and structure your code in an organized and efficient manner.

Here's how you define a simple class in Python:

```
# Example of a simple class
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display_info(self):
        print(f"{self.brand} {self.model}")
```

→ The __init__ method (short for *initialize*) is a special method in Python classes. It is called when an object is created and allows you to set up its initial attributes.
→ The *self* parameter represents the instance of the class and is used to access the attributes and methods within the class.
→ The display_info method is a simple method that prints information about the car.

Creating Objects (Instances)
Once you have a class, you can create objects (instances) based on that class:

```
# Creating objects from the Car class
car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "Accord")

# Accessing attributes and calling methods
car1.display_info()  # Output: Toyota Camry
car2.display_info()  # Output: Honda Accord

// Output
>> Toyota Camry
>> Honda Accord
```

In this example, car1 and car2 are **instances** of the Car class. Each instance has its own set of attributes (brand and model) and can call the methods defined in the class.

Attributes and Methods
Attributes are like variables that belong to a class, and methods are functions that belong to a class. They allow you to **encapsulate** data and behavior within a class.

```
# Example of using attributes and methods
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print("Woof!")
```

```
# Creating an object
my_dog = Dog("Buddy", 3)

# Accessing attributes
print(f"{my_dog.name} is {my_dog.age} years old.")

# Calling a method
my_dog.bark()

// Output
>> Buddy is 3 years old.
>> Woof!
```

Here, the Dog class has attributes (name and age) and a method (bark). The object my_dog has its own values for these attributes and can call the bark method.

---

13. Recursion

Recursion is a programming paradigm where a function calls itself in its own definition.

Recursive Function Example - Factorial:
An example of recursion is calculating the factorial of a number. The factorial of a non-negative integer n, denoted as n!, is the product of all positive integers less than or equal to n.

```
# Recursive function example: Factorial
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)

# Calculating factorial
result = factorial(5)
print(f"The factorial of 5 is {result}.")

// Output
>> The factorial of 5 is 120.
```

In the example above, the factorial function calculates the factorial of a number using recursion. The base case (n == 1) prevents an infinite loop, and the function gradually breaks down the problem into smaller sub-problems. Understanding recursion is like solving a puzzle - it's about breaking a big problem into smaller, more manageable pieces.

In every recursive function there is a base case and a recursive case:

- Base Case: The base case is the **condition under which the recursion stops**. It prevents the function from calling itself indefinitely. In the factorial example, the base case is when n=0 or n=1, and the function returns 1.

- Recursive Case: The recursive case **defines how the function calls itself** with a smaller or simpler input. In the factorial example, the recursive case is n! = n x (n - 1)!

Recursion is particularly useful when a problem can be broken down into smaller, identical sub-problems. However, not all problems are well-suited for recursion, and iterative solutions may be more appropriate in some cases.

---

14. File I/O

Let's step up your file-handling skills. Reading and writing files are essential for more complex programs.

1) Reading from a File:

```
with open("example.txt", "r") as file:
    content = file.read()
    print(f"Content of the file:\n{content}")
```

- open("example.txt", "r") | Opens the file named "example.txt" in read mode ("r").
- with ... as file: | Uses the with statement to ensure proper handling of the file.
    - The file is automatically closed when the block inside the with statement is exited.
- content = file.read() | Reads the entire content of the file and stores it in the variable content.
- print(f"Content of the file:\n{content}") | Prints the content of the file.

2) Writing to a File:

```
with open("output.txt", "w") as file:
    file.write("This is some content.")
```

- open("output.txt", "w") | Opens the file named "output.txt" in write mode ("w").
    - If the file already exists, it will be truncated. If it doesn't exist, a new file will be created.
- with ... as file: | Ensures proper handling of the file.
- file.write("This is some content.") | Writes the specified content to the file.

3) Reading the Content of the Newly Created File:

```
with open("output.txt", "r") as file:
    new_content = file.read()
    print(f"\nContent of the 'output.txt' file:\n{new_content}")
```

Similar to the first example, this reads the content of the "output.txt" file and prints it.

4) Appending to a File:

```
with open("output.txt", "a") as file:
    file.write("\nThis is appended content.")
```

- open("output.txt", "a") | Opens the file in append mode ("a").
    - If the file exists, the new content will be added at the end.
- file.write("\nThis is appended content.") | Appends the specified content to the file.

5) Reading the Updated Content:

```
with open("output.txt", "r") as file:
    updated_content = file.read()
    print(f"\nUpdated content of the 'output.txt' file:\n{updated_content}")
```

Similar to the third example, this reads and prints the updated content of the "output.txt" file.

6) Reading File Line by Line:

```
with open("example.txt", "r") as file:
    lines = file.readlines()
    print("\nContent of the file (line by line):")
    for line in lines:
        print(line.strip())
```

- file.readlines() | Reads all the lines of the file and returns them as a list of strings.
- The for loop iterates through each line and prints it using print(line.strip()). strip() is used to remove the newline character at the end of each line.

7) Using the 'with' Statement for Multiple Files:

```
with open("file1.txt", "r") as file1, open("file2.txt", "r") as file2:
    content1 = file1.read()
    content2 = file2.read()
    print("\nContent of file1.txt:\n", content1)
    print("\nContent of file2.txt:\n", content2)
```

Demonstrates how to use the with statement for multiple files simultaneously (file1 and file2). Reads the content of each file and prints it.

15. Modules (& Packages)

In Python, modules and packages are crucial concepts that contribute to code organization, reusability, and maintainability. Modules allow you to group related code into files, while packages provide a way to structure larger projects by organizing modules into directories.

Here, we will provide some more information about modules and some helpful ones you can try to use!

Module Importing: To use code defined in another file (module), you can use the import statement.

```
# Importing the entire module
import my_module

# Using a function from the module
my_module.my_function()
```

Importing Specific Elements: You can import specific functions or variables from a module.

```
from my_module import my_function

# Using the imported function directly
my_function()
```

Importing with Alias: You can use an alias for a module or specific elements to simplify their usage.

```
import my_module as mm

# Using the alias
mm.my_function()
```

Creating Modules: A module is simply a Python file. Create a file (e.g., my_module.py) and define your code in it.

```
# my_module.py
def my_function():
    print("Hello from my_function!")
```

Using a Module: Once a module is created, you can import and use it in other files.

```
import my_module

my_module.my_function()
```

Here are examples of some helpful Python modules and how to use them:

1. math Module:

The math module provides mathematical functions for performing operations like square root, logarithm, trigonometry, etc.

```
import math

# Example 1: Calculate the square root
result = math.sqrt(25)
print(result)  # Output: 5.0

# Example 2: Calculate the cosine of an angle in radians
angle = math.radians(45)
cosine_value = math.cos(angle)
print(cosine_value)  # Output: 0.7071067811865475

// Output
>> 5.0
>> 0.7071067811865475
```

2. datetime Module:

The datetime module helps with date and time manipulation.

```
from datetime import datetime, timedelta

# Example 1: Get the current date and time
current_datetime = datetime.now()
print(current_datetime)

# Example 2: Calculate a future date
future_date = current_datetime + timedelta(days=7)
print(future_date)
```

3. random Module:

The random module is used for generating pseudo-random numbers.

```
import random

# Example 1: Generate a random integer between 1 and 10
random_number = random.randint(1, 10)
print(random_number)

# Example 2: Shuffle a list
my_list = [1, 2, 3, 4, 5]
random.shuffle(my_list)
print(my_list)
```

Glossary & Resources

Word	***Definition / Ex***
Program	A program is a set of instructions that a computer uses to perform a specific action. Sometimes it's compared to a recipe with variables as ingredients and functions as instructions. Applications (like your browser) are a type of program.
Assignment → Declaration / Declaring → Initialization / Initializing	Declaration declares the creation of variables and functions. ● Declaration is not concerned with the storage (initialization) of values. Instead, its job is to declare the creation of variables and functions. Initialization assigns initial values to variables. In Python, declaration and initialization are combined in a process called assignment. Assignment means "this name on the left-hand side now refers to the result of evaluating the right-hand side, regardless of what it referred to before (if anything)". person1_name = “Kat” Here we assigned “Kat” to the variable person1_name.
Index / Indexing	Indexing refers to the process of accessing a specific element in a sequence, such as a string or list, using its position or index number. Indexing in Python starts at 0, which means that the first element in a sequence has an index of 0, the second element has an index of 1, and so on.
Iteration	In programming, you will define several steps to achieve specific tasks. In some instances, you will need to continue those steps for *n times* until it meets a particular condition. In these cases, you will make an iterative loop - where one round of it is an iteration. This loop repeats a block of code until a specific result occurs.
Sequential Execution	Where statements are always performed one after the next, in exactly the order specified.